AI_ASSISTED_10.1 ASSIGNMENT

NAME: Mohd Manzoor Ahmed
BATCH:46 HT.NO:2303A53042

Lab Objectives

- Use AI for automated code review and quality enhancement.
- Identify and fix syntax, logical, performance, and security issues in Python code.
- Improve readability and maintainability through structured refactoring and comments.
- Apply prompt engineering for targeted improvements.
- Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices

Lab Outcomes

1. Students will be able to use AI tools to review code.
2. Students will be able to improve code quality and readability.
3. Students will be able to identify and fix common coding issues.

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student
    def calc_average(marks):
```

```
    total = 0
    for m in marks:
    total += m


    average = total / len(marks)
    return avrage   # Typo here


marks = [85, 90, 78, 92]


print("Average Score is ", calc_average(marks)
```

Expected Output:

- Corrected and runnable Python code with explanations of the

fixes.

Prompt:

Identify and fix syntax and logical errors in the following Python code and provide corrected runnable code.

Code:

```
def calc_average(marks):
   total = 0     for
m in marks:
       total += m
   average       =       total       /
len(marks)     return average
marks = [85, 90, 78, 92]
print("Average Score is", calc_average(marks))
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20)) Expected
```

Output:

- Well-formatted PEP 8-compliant Python code.

Prompt:

Refactor the following Python code to follow PEP 8 style guidelines and improve readability.

Code:

```
# Function to calculate area of a rectangle
def area_of_rectangle(length, breadth):
    return length * breadth
print("Area of Rectangle:", area_of_rectangle(10, 20))
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
 return x*y/100
a=200
b=15
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

Prompt:

Improve the readability of the following Python code by using meaningful names and proper formatting without changing its logic.

Code:

```
def calculate_percentage(amount, percentage):
   return amount * percentage / 100
principal_amount = 200
interest_rate = 15

result = calculate_percentage(principal_amount, interest_rate)

print("Calculated Percentage Value:", result)
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

	students = ["Alice", "Bob", "Charlie"] print("Welcome", students[0]) print("Welcome", students[1]) print("Welcome", students[2]) Expected Output: • Modular code with reusable functions.	

Prompt:

Refactor the following repetitive Python code into reusable functions to improve maintainability.

Code:

```
def welcome_students(student_list):
for student in student_list:
    print("Welcome", student)

students = ["Alice", "Bob", "Charlie"]
welcome_students(students)
```

Task Description #5 – Performance Optimization Task:

Use AI to make the code run faster.

Sample Input Code:

```
# Find squares of numbers
        nums = [i for i in range(1,1000000)]
        squares = [] for n in nums:
          squares.append(n**2)
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

Prompt:

Optimize the following Python code to improve performance and memory efficiency.

Code:

```
nums = range(1, 1_000_000)
squares = [n ** 2 for n in nums]

print(len(squares))
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Prompt:

Simplify the following nested conditional logic to make the code cleaner

	and more readable. **Code:** def grade(score): if score >= 90: return "A" elif score >= 80: return "B" elif score >= 70: return "C" elif score >= 60: return "D" else: return "F"	